

DIAGNÓSTICO DE FERRAMENTAL E CONFRONTO DE PARADIGMAS

Ferramentas CASE e Fundamentos do Desenvolvimento Orientado a Objetos



UNASP SP - Projeto de Software (G00264.1)

Atividade - PDF único + repositório Git

Data da atividade: 13/08/2026

Integrante: Nicolas Magalhães - RA: 212907

Repositório Git: <https://github.com/AlnurTV/atividade-case-unasp>

PARTE A Inventário e classificação do ferramental

Foram selecionadas seis ferramentas instaladas ou acessíveis via navegador que apoiam diferentes etapas do ciclo de vida do software. As classificações seguem a taxonomia apresentada na aula: Upper CASE, Lower CASE e Integrated CASE (I-CASE).

A.1 e A.2 - Levantamento e classificação das ferramentas

Ferramenta	Versão	Categoria CASE	Tipo funcional	Etapas do ciclo de vida
draw.io / diagrams.net	Web (versão contínua)	Upper CASE	Diagramação	Requisitos, análise e projeto
StarUML	7.1.0	Upper CASE	Modelagem UML / análise e projeto	Análise e projeto orientados a objetos
MySQL Workbench	8.0.47	Upper CASE	Modelagem de dados	Projeto de dados
Visual Studio Code	1.134.0	Lower CASE	Codificação / IDE	Construção e implementação
Git	2.55.0.windows.4	Integrated CASE (I-CASE)	Controle de versão / repositório	Gestão de configuração
GitHub	Web / SaaS (sem versão fixa)	Integrated CASE (I-CASE, em conjunto com Git)	Repositório remoto / colaboração	Gestão de configuração

Observação sobre versões Web

draw.io/diagrams.net e GitHub são serviços web com atualização contínua. Por isso, quando não há um número de versão fixo exposto ao usuário, a versão é registrada como Web/SaaS, sem inventar um build local.

A.3 Toolchain escolhido para o semestre

Toolchain selecionado: StarUML 7.1.0 + Visual Studio Code 1.134.0 + Git 2.55.0.windows.4, utilizando o GitHub como repositório remoto. A combinação cobre modelagem, implementação e gestão de configuração, em linha com o mapa operacional apresentado na aula.

StarUML 7.1.0

O StarUML foi escolhido para as etapas de análise e projeto orientados a objetos, pois permite criar e manter diagramas UML que representam classes, atributos, métodos e relacionamentos antes da implementação. Como benefício discutido em aula, destaca-se a aderência a padrões e qualidade: a UML padroniza a comunicação do modelo ao longo do projeto. Como risco, existe a curva de treinamento, pois a ferramenta exige domínio das notações UML e de seus recursos para ser utilizada corretamente.

Visual Studio Code 1.134.0

O Visual Studio Code foi escolhido para a construção e implementação do software. Ele permite editar, organizar e executar o código do projeto, além de receber extensões conforme as necessidades do trabalho. Como benefício, contribui para maior produtividade e menor retrabalho na etapa de construção. Como risco, há o perigo de expectativas irreais: o ambiente não substitui a competência do desenvolvedor nem corrige automaticamente decisões ruins de análise ou projeto.

Git 2.55.0.windows.4

O Git foi escolhido para a gestão de configuração e o versionamento dos artefatos do projeto, com o GitHub utilizado como repositório remoto para colaboração e entrega. Como benefício discutido em aula, favorece a documentação consistente e o histórico controlado de modelos, código e documentos. Como risco, existe a resistência cultural: se boas práticas de commits e organização forem abandonadas sob pressão de prazo, o controle de versões perde confiabilidade.

PARTE B Modelagem do mesmo caso em duas visões

Mini-caso: Sistema de Empréstimo de Equipamentos do Laboratório

O laboratório precisa controlar empréstimos de notebooks, projetores e kits de robótica para alunos e professores. Cada empréstimo registra solicitante, equipamento, data de retirada e data prevista de devolução. Alunos podem retirar um item por vez, por até dois dias; professores podem retirar até três itens, por até sete dias. Devoluções em atraso geram uma penalidade que bloqueia novos empréstimos por quinze dias.

Premissas adotadas

- Cada solicitante possui um identificador único.
- Cada equipamento possui um identificador único.
- Um equipamento só pode estar em um empréstimo ativo por vez.
- A devolução realizada após a data prevista caracteriza atraso.
- O bloqueio de 15 dias começa na data em que o equipamento atrasado é devolvido.
- Durante o bloqueio, o solicitante não pode realizar novos empréstimos.
- Para o escopo desta atividade, o status operacional do equipamento é tratado como Disponível ou Emprestado.

B.1 Visão estruturada - DFD nível 0

O nível 0 apresenta o sistema como um único processo e mostra as trocas de dados com a entidade externa Solicitante (Aluno ou Professor).

DFD NÍVEL 0 - DIAGRAMA DE CONTEXTO

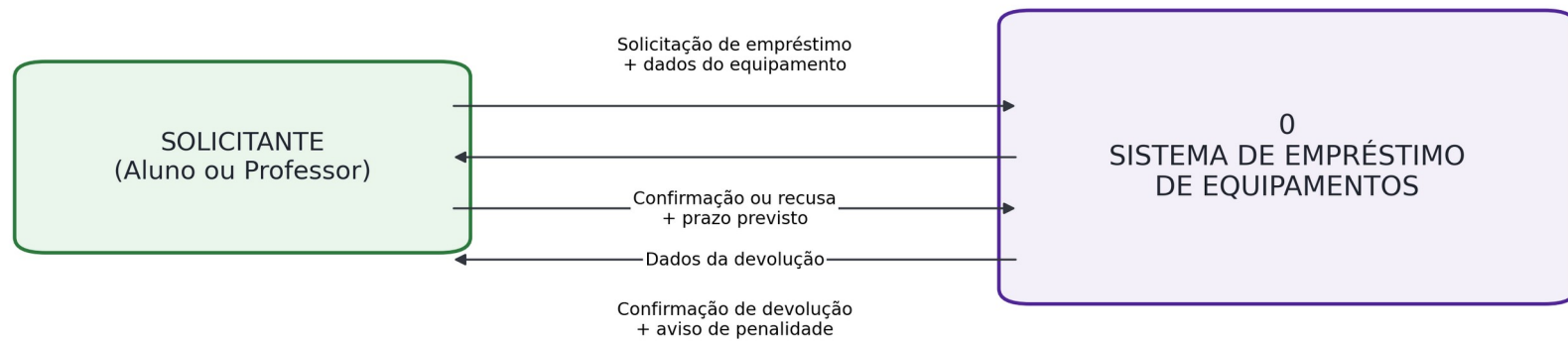


Figura 1 - DFD nível 0 (diagrama de contexto).

B.1 Visão estruturada - DFD nível 1

O nível 1 decompõe o sistema em quatro processos e utiliza quatro depósitos de dados: solicitantes, equipamentos, empréstimos e penalidades.

DFD NÍVEL 1 - DECOMPOSIÇÃO FUNCIONAL

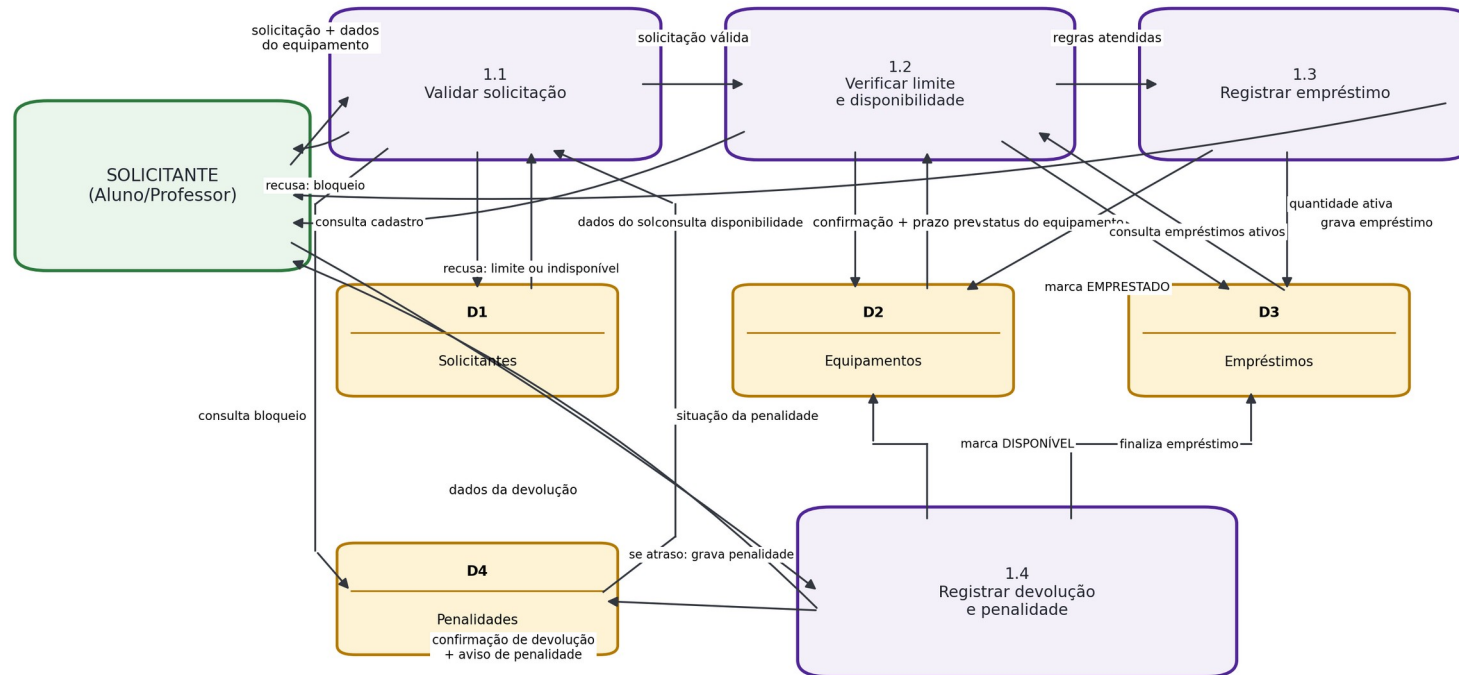


Figura 2 - DFD nível 1 (decomposição funcional).

B.1 Dicionário de dados

Elemento	Tipo	Domínio / valores permitidos	Descrição
idSolicitante	Inteiro	Número positivo	Identificador único do solicitante
nomeSolicitante	Texto	Até 100 caracteres	Nome do aluno ou professor
tipoSolicitante	Enumeração	Aluno Professor	Define as regras de empréstimo aplicáveis
idEquipamento	Inteiro	Número positivo	Identificador único do equipamento
tipoEquipamento	Enumeração	Notebook Projetor Kit de Robótica	Categoria do equipamento
statusEquipamento	Enumeração	Disponível Emprestado	Indica se o item pode ser retirado
idEmprestimo	Inteiro	Número positivo	Identificador único do empréstimo
dataRetirada	Data	Data válida	Data efetiva da retirada
dataPrevistaDevolucao	Data	Data válida >= dataRetirada	Prazo previsto para devolução
dataDevolucao	Data / Nulo	Data válida ou vazio	Data efetiva da devolução
statusEmprestimo	Enumeração	Ativo Finalizado Atrasado	Situação do empréstimo
quantidadeItensAtivos	Inteiro	Aluno: 0-1 Professor: 0-3	Quantidade de itens atualmente emprestados
prazoMaximo	Inteiro	Aluno: 2 dias Professor: 7 dias	Prazo máximo conforme o tipo de solicitante
possuiPenalidade	Booleano	Verdadeiro Falso	Indica se existe bloqueio ativo
idPenalidade	Inteiro	Número positivo	Identificador único da penalidade
dataInicioPenalidade	Data / Nulo	Data válida ou vazio	Início do bloqueio por atraso
dataFimPenalidade	Data / Nulo	15 dias após o início	Fim do período de bloqueio

B.2 Visão orientada a objetos - Diagrama de classes

O modelo OO organiza o domínio em classes. Solicitante é abstrata; Aluno e Professor especializam as regras de quantidade e prazo. Equipamento, Empréstimo e Penalidade representam os demais conceitos centrais.

DIAGRAMA DE CLASSES - SISTEMA DE EMPRÉSTIMO DE EQUIPAMENTOS

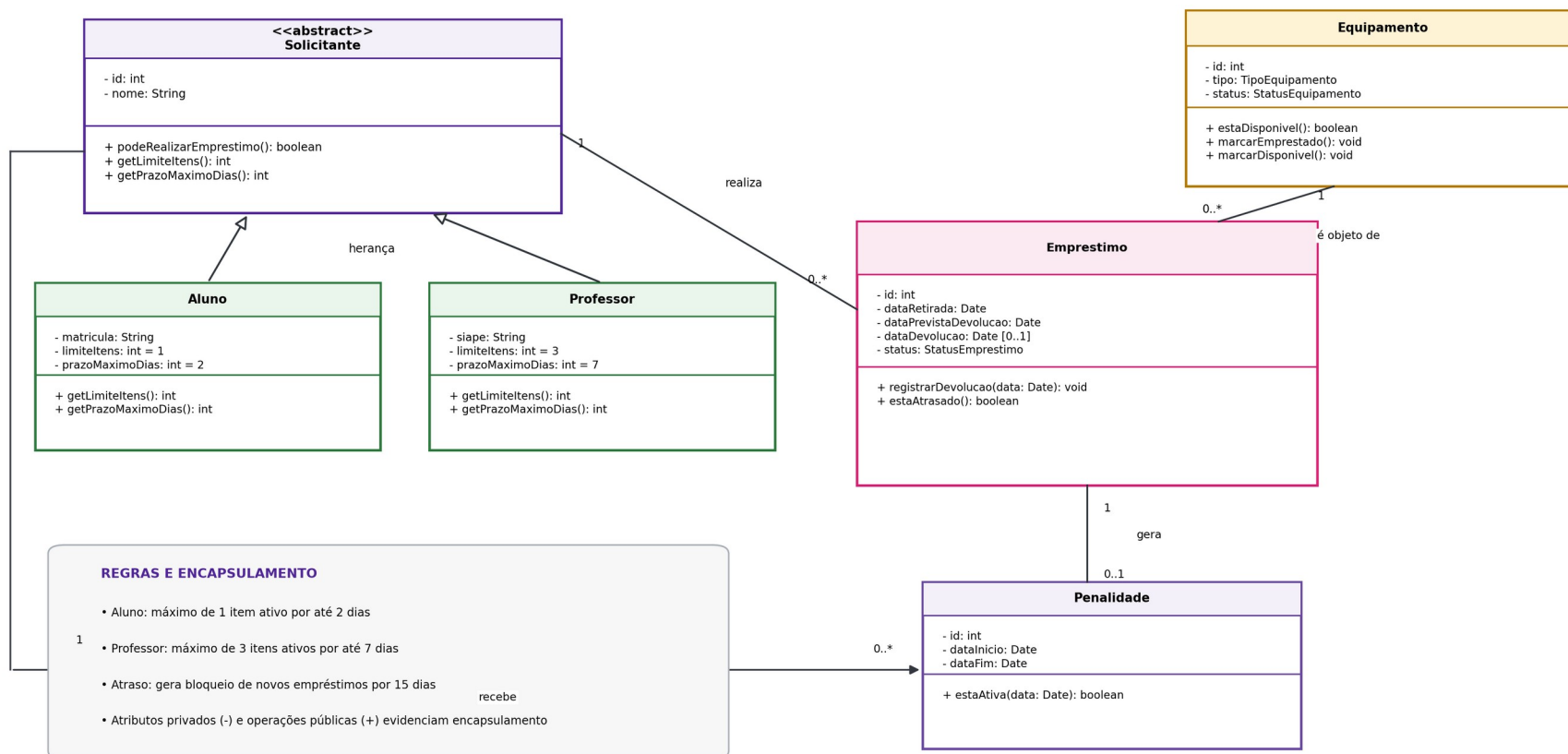


Figura 3 - Diagrama de classes UML do sistema de empréstimos.

B.2 Encapsulamento e herança

Encapsulamento

Os atributos das classes são privados, identificados pelo símbolo “-” no diagrama. O estado interno é alterado por métodos públicos, como `estaDisponivel()`, `registrarDevolucao()` e `estaAtiva()`. Assim, as regras permanecem próximas dos dados pelos quais são responsáveis e evita-se a alteração direta e inconsistente do estado dos objetos.

Herança

Aluno e Professor herdam de Solicitante. A superclasse reúne id, nome e operações comuns; as subclasses especializam `getLimitetens()` e `getPrazoMaximoDias()`. Isso evita duplicação e expressa diretamente as regras do enunciado: aluno = 1 item por até 2 dias; professor = até 3 itens por até 7 dias.

Regra de penalidade

Quando `registrarDevolucao()` identifica uma devolução após a data prevista, o sistema cria uma Penalidade com duração de 15 dias. Enquanto a penalidade estiver ativa, `podeRealizarEmprestimo()` deve retornar falso.

B.3 Confronto entre os dois modelos

Para o Sistema de Empréstimo de Equipamentos, o paradigma orientado a objetos apresenta maior adequação do que o paradigma estruturado. A conclusão foi construída a partir de quatro critérios: unidade de decomposição, relação entre dados e comportamento, acoplamento entre as partes e custo de manutenção/tratamento de mudanças.

1. Unidade de decomposição

No modelo estruturado, o sistema é decomposto em processos, como validar a solicitação, verificar limite e disponibilidade, registrar empréstimo e registrar devolução/penalidade. No modelo OO, a decomposição ocorre por abstrações do domínio, representadas por classes como Solicitante, Aluno, Professor, Equipamento, Empréstimo e Penalidade. Para este caso, as classes aproximam o modelo da realidade do laboratório e tornam as responsabilidades mais explícitas.

2. Relação entre dados e comportamento

No DFD, os dados circulam entre processos e depósitos separados. No modelo OO, atributos e operações relacionadas permanecem dentro das próprias classes. Equipamento controla sua disponibilidade; Empréstimo controla datas e atraso; Penalidade controla o período de bloqueio. O encapsulamento reduz a exposição direta dos dados e concentra as regras no elemento responsável.

3. Acoplamento entre as partes

Na visão estruturada, vários processos consultam ou atualizam os mesmos depósitos de dados. Uma mudança na estrutura do empréstimo pode exigir revisão de diferentes processos. Na visão OO, as classes colaboram por interfaces e métodos definidos, diminuindo a dependência direta entre as partes e permitindo que cada classe preserve sua própria consistência.

4. Custo de manutenção e tratamento da mudança

Se surgir um novo tipo de solicitante com regras específicas, o modelo estruturado exigiria revisar fluxos e validações em vários pontos. No modelo OO, pode-se criar uma nova especialização de Solicitante e implementar seus limites e prazos, preservando a maior parte da estrutura existente. Portanto, para um sistema sujeito a evolução de regras, a abordagem orientada a objetos oferece maior organização, menor impacto de mudanças e melhor capacidade de manutenção.

Conclusão

Embora o paradigma estruturado represente com clareza o fluxo de transformação das informações, o paradigma orientado a objetos atende melhor este mini-caso porque o domínio possui entidades com responsabilidades e regras próprias, além de apresentar maior flexibilidade para futuras mudanças.

Base conceitual

Modelagem e classificações elaboradas a partir do material da Aula 03 de Projeto de Software - Ferramentas CASE e os Fundamentos do Desenvolvimento Orientado a Objetos (UNASP SP, 06/08/2026).